

Artificial Intelligence & Machine Learning– Task 2

Feature Engineering, Model Optimization & Performance Comparison

Objective

In **Task 1**, you learned how to:

- Load a dataset
- Train a basic Machine Learning model
- Evaluate model performance

In **Task 2**, you will learn how **Machine Learning** engineers improve models in real-world projects.

This task focuses on:

- Preparing data correctly for Machine Learning
- Improving model performance using preprocessing techniques
- Training multiple algorithms
- Comparing results and selecting the best-performing model

By the end of this task, you will understand how professional ML workflows go beyond basic model training.

Why This Task Matters

In real-world Machine Learning projects:

- Models are rarely deployed after the first attempt
- Data is cleaned, transformed, and refined multiple times
- Different algorithms are tested and evaluated
- Decisions are made using measurable performance metrics

This task trains you to:

- Think analytically about model quality
- Justify technical decisions using data
- Follow an industry-aligned Machine Learning pipeline

What You Will Build

You will build an **enhanced House Price Prediction system** that includes:

- Feature-scaled input data
- Multiple regression models
- A structured model performance comparison
- A final selected model with proper justification

This project is suitable for professional portfolios and technical interviews.

Concepts You Will Learn

Concept	Purpose
Feature Scaling	Ensures fair learning across features
Model Comparison	Identifies the best-performing algorithm
Evaluation Metrics	Measures real model performance
Overfitting (Introduction)	Prevents misleading accuracy
ML Workflow	Mirrors real industry practices

Dataset Used

You will use the **California Housing Dataset**, which contains real housing-related data.

Target variable:

- Median House Value

Input features include:

- Median Income
 - House Age
 - Average Rooms
 - Population and location-based attributes
-

Tools & Technologies

- Python
- Jupyter Notebook
- pandas
- NumPy
- scikit-learn
- matplotlib / seaborn

Step-by-Step Implementation

Step 1: Import Required Libraries

```
python

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

These libraries handle data manipulation, model training, evaluation, and visualization.

Step 2: Load the Dataset

```
python

data = fetch_california_housing(as_frame=True)
df = pd.concat([data.data, data.target.rename("HousePrice")], axis=1)

df.head()
```

At this stage, review column names, data types, and the target variable.

Step 3: Separate Features and Target Variable

```
python
```

[Copy code](#)

```
X = df.drop("HousePrice", axis=1)
y = df["HousePrice"]
```

Machine Learning models require input features (X) and an output variable (y) to be handled separately.

Step 4: Feature Scaling (Critical Step)

Features in the dataset exist on different numeric ranges.

Without scaling:

- Some features dominate others
- Model performance becomes unstable

```
python
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

After scaling, all features have a common scale, improving learning stability and performance.

Step 5: Train-Test Split

```
python
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

This ensures the model is evaluated using unseen data rather than memorized values.

Step 6: Train Multiple Models

Instead of relying on a single algorithm, you will train and evaluate multiple models.

```
python
```

```
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Decision Tree": DecisionTreeRegressor(max_depth=5)
}
```

Model rationale:

- Linear Regression serves as a baseline
 - Ridge Regression helps reduce overfitting
 - Decision Tree captures non-linear relationships
-

Step 7: Model Evaluation and Comparison

```
python

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    rmse = mean_squared_error(y_test, predictions, squared=False)
    r2 = r2_score(y_test, predictions)

    results[name] = {
        "RMSE": rmse,
        "R2 Score": r2
    }

results_df = pd.DataFrame(results).T
results_df
```

Interpretation:

- Lower RMSE indicates better prediction accuracy
- Higher R^2 score indicates better explanatory power

Step 8: Visual Performance Validation

```
python

best_model = LinearRegression()
best_model.fit(X_train, y_train)

y_pred = best_model.predict(X_test)

plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.4)
plt.xlabel("Actual House Prices")
plt.ylabel("Predicted House Prices")
plt.title("Actual vs Predicted House Prices")
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color="red")
plt.show()
```

A closer alignment of points to the reference line indicates better predictive performance.

Key Learning Outcomes

After completing Task 2, you will be able to:

- Apply preprocessing techniques correctly
 - Compare multiple Machine Learning models objectively
 - Interpret evaluation metrics professionally
 - Explain and justify model selection decisions
-

Deliverables

Mandatory:

1. Jupyter Notebook named `AI_ML_Task2_Model_Comparison.ipynb`
2. Model performance comparison table
3. A 1–2 page PDF explaining methodology, results, and conclusions

Optional:

- Save the best-performing model using `joblib`
- Add additional charts or performance insights

Free Learning Resources & Support:

1. Python for Machine Learning (Revision Support)

These resources help interns who may feel slightly weak in Python basics.

Free YouTube Courses

- **Python for Beginners – Full Course** (freeCodeCamp)
<https://www.youtube.com/watch?v=rfscVS0vtbw>
- **Python for Data Science** (Simplilearn – Free)
<https://www.youtube.com/watch?v=LHBE6Q9Xlzl>

Reference

- Python Official Documentation
<https://docs.python.org/3/>

2. Data Analysis with pandas & NumPy

Required for:

- Dataset handling
- Feature inspection
- Data preparation

Free YouTube Tutorials

- **pandas Tutorial for Beginners** (Corey Schafer)
<https://www.youtube.com/watch?v=vmEHCJofslg>
- **NumPy Full Course** (freeCodeCamp)
<https://www.youtube.com/watch?v=QUT1VHiLmml>

Official Documentation

- pandas Documentation
<https://pandas.pydata.org/docs/>
- NumPy Documentation
<https://numpy.org/doc/>

3. Feature Scaling & Data Preprocessing (Very Important)

This directly supports Task-2 Step-4 (Feature Scaling).

Free YouTube Videos

- Feature Scaling Explained Simply
<https://www.youtube.com/watch?v=mnKm3YP56PY>
- StandardScaler vs MinMaxScaler
<https://www.youtube.com/watch?v=ksx8ttH8cN4>

Reference

- scikit-learn Preprocessing Guide
<https://scikit-learn.org/stable/modules/preprocessing.html>

4. Machine Learning Models (Regression Focus)

Covers:

- Linear Regression
- Ridge Regression
- Decision Tree Regressor

Free YouTube Courses

- Machine Learning Full Course (Andrew Ng – Free Audit concepts)
<https://www.youtube.com/watch?v=PPLop4L2eGk>
- Linear Regression Explained Clearly
<https://www.youtube.com/watch?v=ZkjP5RJLQF4>
- Decision Trees Explained
<https://www.youtube.com/watch?v=7VeUPuFGJHk>

Official Documentation

- Linear Regression (scikit-learn)
https://scikit-learn.org/stable/modules/linear_model.html
- Decision Tree Regressor
<https://scikit-learn.org/stable/modules/tree.html>

5. Model Evaluation Metrics (RMSE, R^2)

This section is critical for Task-2 evaluation & comparison.

Free YouTube Resources

- RMSE, MAE, R^2 Explained Clearly
<https://www.youtube.com/watch?v=ZkhC8Xr5fH8>
- How to Evaluate ML Models
<https://www.youtube.com/watch?v=85dtiMz9tSo>

Reference

- Model Evaluation in scikit-learn
https://scikit-learn.org/stable/modules/model_evaluation.html

6. Dataset Resources (Used in Task)

Primary Dataset

- California Housing Dataset (Official)
https://scikit-learn.org/stable/datasets/real_world.html

Optional Practice Source

- Kaggle – California Housing Dataset
<https://www.kaggle.com/datasets/camnugent/california-housing-prices>

(Kaggle is optional; dataset is already available via scikit-learn.)

7. Visualization Support (Graphs & Plots)

Used for:

- Actual vs Predicted plots
- Understanding model behavior

Free YouTube Tutorials

- matplotlib for Beginners
<https://www.youtube.com/watch?v=UO98IIQ3QGI>
- Data Visualization for ML
<https://www.youtube.com/watch?v=0P7QnIQDBJY>

Reference

- matplotlib Documentation
<https://matplotlib.org/stable/contents.html>
-

8. Jupyter Notebook & Environment Setup

Free Tutorials

- Jupyter Notebook Beginner Guide
<https://www.youtube.com/watch?v=HW29067qVWk>
- Anaconda Installation & Setup
<https://www.youtube.com/watch?v=YJC6ldI3hWk>

Tools

- Anaconda (Free)
<https://www.anaconda.com/download>
-

9. Additional Support (Highly Recommended)

Concept Reinforcement

- Machine Learning Crash Course (Google – Free)
<https://developers.google.com/machine-learning/crash-course>

Practice & Revision

- freeCodeCamp ML Articles
<https://www.freecodecamp.org/news/tag/machine-learning/>
-

Summary for Interns

To complete AI/ML Task–2 successfully, an intern should:

- Watch 2–3 YouTube videos per section
- Follow official documentation for reference
- Implement everything in a Jupyter Notebook
- Focus on understanding, not memorizing

No paid course is required to complete this task professionally.